SIT

Siddaganga Institute of Technology Tumakuru - 572 103, Karnataka

Edge-Enabled Privacy-Preserving System for Construction Workspace Management

Chapter 6
Software Requirements, Algorithm & Flowchart

Mini-Project Report | Academic Year 2026 - 27
Department of ECE / CSE | Siddaganga Institute of Technology, Tumakuru

Team Members
Mahalakshmi R
Prathibha V
Priyanka S
Anurag A Aralimarad
Badagi Lokamurthy

CHAPTER 6

Software Requirements, Algorithm & Flowchart

This chapter documents the complete software requirements for the Edge-Enabled Privacy-Preserving Construction Safety System, describes the algorithmic logic that drives safety detection and alerting, and presents the system flowcharts illustrating data and control flow from camera input through to voice alert delivery.

6.1 About Software Requirements

The system is architected as a two-tier application (backend + frontend) deployed entirely on a local edge device (Raspberry Pi 4). Both tiers communicate over a local-network REST API. The tables below detail every software dependency, its purpose, minimum compatible version, and licence.

6.1.1 Hardware Requirements

Component	Specification	Purpose
Processing Unit	Raspberry Pi 4 Model B (4 GB RAM)	Edge inference, local web server, database host
Camera	USB Webcam / Pi Camera V2 / AR/VR Headset	Real-time frame capture of construction workspace
Storage	16 GB+ Class 10 MicroSD (A1 rated)	OS, application code, SQLite database, YOLO model weights
Network	Ethernet / Wi-Fi (site LAN only)	Frontend access from on-site devices; no internet required
Audio Output	3.5 mm speaker or USB audio adapter	Voice alert playback via pyttsx3/espeak
Power Supply	5 V / 3 A USB-C PSU (UPS HAT recommended)	Continuous operation at construction sites
Development Machine	Intel i5+ / 8 GB RAM / Windows 10+	Development, testing, model training (CPU or GPU)

Table 6.1 -- Hardware requirements specification.

6.1.2 Backend Software Requirements (Python)

The backend is implemented entirely in Python and runs as a FastAPI ASGI server. Table 6.2 lists all production dependencies with their roles.

Package	Version	Licence	Role in System
Python	>= 3.10	PSF	Runtime interpreter
FastAPI	>= 0.110	MIT	REST API framework; async ASGI server
Uvicorn	>= 0.28	BSD-3	ASGI server (production-grade, multi-worker)
Pydantic	>= 2.6	MIT	Request/response validation; Settings model
SQLAlchemy	>= 2.0	MIT	ORM for SQLite; models, sessions, queries
SQLite	3.x (std-lib)	Public Domain	Local edge database; zero-config
OpenCV-Python	>= 4.8	Apache 2.0	Image capture, preprocessing, CLAHE, blur, annotation
NumPy	>= 1.24	BSD-3	Array operations for image and HSV analysis
Pillow	>= 10.0	HPND	Supplementary image I/O

Package	Version	Licence	Role in System
Ultralytics	>= 8.1	AGPL-3	YOLOv8 model loading, inference, export APIs
PyTorch	>= 2.0	BSD-3	Deep-learning backend for YOLO inference
TorchVision	>= 0.15	BSD-3	Image transforms (required by Ultralytics)
pyttsx3	>= 2.90	MPL-2.0	Offline text-to-speech for voice safety alerts
python-multipart	>= 0.0.9	Apache 2.0	File upload handling (video samples)
pytest	>= 8.0	MIT	Unit and integration test framework
HTTPX	>= 0.27	BSD-3	Async test client for FastAPI endpoints

Table 6.2 -- Backend software dependencies.

6.1.3 Frontend Software Requirements (React)

The frontend is a single-page application (SPA) built with React 18 and bundled by Vite. It communicates with the backend exclusively via Axios HTTP calls. No AI logic, database operations, or camera access exists in the frontend -- maintaining strict separation of concerns.

Package	Version	Licence	Role in System
Node.js	>= 18.x	MIT	JavaScript runtime for build tooling
React	18.3	MIT	Component-based UI framework
React DOM	18.3	MIT	Browser rendering engine for React
React Router DOM	6.29	MIT	Client-side routing (7 pages)
Vite	6.1	MIT	Lightning-fast HMR dev server and production bundler
Tailwind CSS	3.4	MIT	Utility-first CSS framework for responsive design
Axios	1.7	MIT	Promise-based HTTP client for REST API calls
Recharts	2.15	MIT	Composable charting library (bar, pie, area, line)
Lucide React	0.475	ISC	Icon library (safety, status, navigation icons)
PostCSS	8.5	MIT	CSS transformer (Tailwind integration)
Autoprefixer	10.4	MIT	Vendor prefix injection for cross-browser CSS

Table 6.3 -- Frontend software dependencies.

6.1.4 Operating System and Deployment Requirements

Environment	OS / Platform	Notes
Development	Windows 10/11 (x64)	Full development, testing, model training
Edge Deployment	Raspberry Pi OS Lite 64-bit (Bookworm)	Headless; systemd autostart; espeak TTS driver
Browser Client	Any modern browser (Chromium / Firefox / Edge)	Access dashboard via site LAN IP

Table 6.4 -- Operating system and deployment platforms.

6.1.5 Functional Requirements Summary

ID	Requirement	Description
FR-01	Camera Ingestion	Capture frames from USB webcam, Pi Camera V2, uploaded video, or synthetic scene generator.
FR-02	Image Preprocessing	Apply CLAHE contrast enhancement and letterbox resize to normalise frames for YOLO input.
FR-03	Worker Detection	Detect and localise construction workers using YOLOv8 person detection or skin-tone contour analysis.
FR-04	PPE Detection	Identify safety helmets and high-visibility vests via YOLO class labels or HSV anatomical region analysis.
FR-05	PPE-Worker Association	Associate detected PPE items with individual workers using IoU-based spatial containment logic.
FR-06	Violation Analysis	Evaluate each worker against configurable safety rules (missing helmet, missing vest, zone intrusion).
FR-07	Alert Debounce	Suppress duplicate alerts for the same worker and violation type within a configurable cooldown window (default 6 s).
FR-08	Voice Alert Generation	Convert violation warnings to speech in the configured language (EN, HI, KN, ES) using pyttsx3.
FR-09	Safety Recommendations	Return multilingual corrective-action text for each violation type.
FR-10	Local Database Persistence	Store safety events, rules, and audit log entries in SQLite; support purge by retention period.
FR-11	Privacy Safeguards	Blur worker faces before display/storage; never store raw video; never transmit to cloud.
FR-12	MJPEG Live Stream	Encode annotated frames as MJPEG and serve via HTTP for browser-based live monitoring.
FR-13	REST API	Expose 14 endpoints for system control, data retrieval, configuration, and privacy auditing.
FR-14	Web Dashboard	Provide 7 React pages: Dashboard, Live Monitoring, Alerts, History, Analytics, Safety Info, Settings.

Table 6.5 -- Functional requirements specification.

6.1.6 Non-Functional Requirements

ID	Quality Attribute	Requirement
NFR-01	Latency	End-to-end frame-to-alert latency < 200 ms on dev machine (CPU); < 400 ms on Raspberry Pi 4.
NFR-02	Throughput	>= 10 FPS on dev machine; >= 2.5 FPS on Raspberry Pi 4.
NFR-03	Privacy	No raw video storage; no cloud transmission; facial anonymisation; GDPR data-minimisation principles.
NFR-04	Availability	Autostart via systemd; graceful restart on crash; UPS-recommended power.
NFR-05	Maintainability	Modular backend (9 independent packages); all dependencies pinned; documented REST API.
NFR-06	Portability	Runs on Windows (dev), Linux ARM64 (Pi 4), and any Chromium-based browser (frontend).

Table 6.6 -- Non-functional requirements.

6.2 About Algorithm

The core intelligence of the system is a multi-stage edge AI pipeline that processes each video frame through a sequence of well-defined algorithmic steps. This section describes each algorithm in detail, including its inputs, outputs, computational complexity, and implementation rationale.

6.2.1 Edge Safety Detection Pipeline (Main Algorithm)

The main processing loop runs as a background daemon thread (VideoProcessor._processing_loop). Each iteration performs the following steps:

Step 1: Frame Capture

Read the next BGR frame from the active camera source (USB webcam, uploaded video file, or synthetic scene generator). If no frame is available (camera disconnected or end-of-video), sleep 50 ms and retry. The synthetic mode generates animated construction-site scenes for demonstration when no physical camera is attached.

Step 2: CLAHE Contrast Enhancement

Convert the BGR frame to LAB colour space. Apply Contrast-Limited Adaptive Histogram Equalisation (CLAHE) with clipLimit=2.0 and tileGridSize=(8, 8) to the L (luminance) channel. Merge back and convert to BGR. This improves detection robustness under variable outdoor construction lighting. Time complexity: O(W x H) where W, H are frame dimensions.

Step 3: YOLOv8 Object Detection

Pass the enhanced frame to the YOLOv8n model via ultralytics predict() with confidence threshold 0.40, IoU threshold 0.45, and input size 640x640. The model returns bounding boxes, class IDs, and confidence scores. 'person' / 'worker' detections are classified as workers; 'helmet', 'hardhat', 'vest', 'jacket' detections are classified as equipment. Inference complexity: O(N) where N = number of grid cells x anchors (architecture-dependent).

Step 4: HSV Anatomical PPE Verification

When the YOLO model detects workers but no PPE classes (as with the generic COCO-trained YOLOv8n), a colour-based verification algorithm activates:

a) Head Region (top 25% of worker box): Convert to HSV. Apply inRange() masks for yellow [20,80,100]-[35,255,255], orange [5,120,120]-[18,255,255], and white [0,0,190]-[180,40,255] helmets. If the combined mask ratio exceeds 0.14, a 'safety-helmet' detection is emitted.

b) Torso Region (25%-70% of worker box): Apply HSV masks for yellow vest [22,90,100]-[40,255,255] and orange vest [6,120,110]-[19,255,255]. If the combined ratio exceeds 0.18, a 'safety-vest' detection is emitted.

Time complexity: O(W_r x H_r) per worker, where W_r, H_r are region dimensions.

Step 5: PPE-Worker Association

For each detected PPE item, compute the Intersection-over-Union (IoU) and containment ratio with every worker bounding box. Assign the PPE to the worker with the highest IoU (minimum threshold: 0.1). Each worker's record is updated with boolean flags has_helmet and has_vest. Time complexity: O(W x E) where W = workers, E = equipment items.

Step 6: Safety Violation Analysis

For each worker, evaluate against the active safety rule set:

Rule 1: If has_helmet is False -> MISSING_HELMET (severity: HIGH)

Rule 2: If has_vest is False -> MISSING_VEST (severity: MEDIUM)

Rule 3: If worker bounding box overlaps a restricted zone (normalised coordinates) -> RESTRICTED_ZONE_INTRUSION (severity: CRITICAL)

Each violation is checked against the debounce cache: if the same (worker_id, violation_type) pair was

alerted within the cooldown window (default 6 seconds), the alert is suppressed to prevent alarm fatigue.

Step 7: Alert Dispatch and Database Persistence

Violations with trigger_alert=True are: (a) pushed into a ring-buffer alert queue (capacity: 50); (b) inserted into the SQLite safety_events table with violation type, severity, confidence, recommendation, and UTC timestamp; (c) dispatched to the voice TTS engine for spoken alert.

Step 8: Privacy Guard - Facial Blur

If BLUR_FACES_IN_EVENTS is enabled (default: True), the top 28% of each worker bounding box (head region) is Gaussian-blurred with a dynamically sized kernel (ksize = max(15, region_width/3 | 1), sigma=30). This ensures no identifiable facial biometrics appear in the MJPEG stream or any stored frame.

Step 9: Safety Annotation Overlay

Draw visual overlays on the frame: red-tinted semi-transparent rectangles for restricted zones; green bounding boxes for compliant workers; red bounding boxes with violation labels for non-compliant workers; teal/orange boxes for detected equipment items with confidence percentages. FPS and latency counters are overlaid in the top-right corner.

Step 10: MJPEG Encoding and Streaming

Encode the annotated frame as JPEG (quality=80) and store in a thread-safe buffer. The /api/v1/stream endpoint reads this buffer and delivers a multipart/x-mixed-replace MJPEG stream to the browser. The loop then sleeps for max(10 ms, 1/TARGET_FPS - elapsed) to pace at ~15 FPS.

6.2.2 YOLOv8 Object Detection Algorithm

YOLOv8 (You Only Look Once, version 8) is a single-stage, anchor-free object detection architecture developed by Ultralytics. The system uses the YOLOv8n (nano) variant, selected for its optimal balance between accuracy and inference speed on resource-constrained edge hardware.

Architecture Components:

- **Backbone (CSPDarknet53):** Extracts hierarchical feature maps from the input image using cross-stage partial connections that reduce computation while maintaining gradient flow.
- **Neck (PANet + FPN):** Aggregates features at multiple scales via a Path Aggregation Network and Feature Pyramid Network, enabling detection of both large (workers) and small (helmets) objects.
- **Head (Decoupled):** Separate classification and regression heads predict class probabilities and bounding-box coordinates independently, improving convergence.
- **Loss Function:** CIoU (Complete IoU) for bounding-box regression; Binary Cross-Entropy for classification; Distribution Focal Loss for objectness.
- **Post-Processing:** Non-Maximum Suppression (NMS) with IoU threshold 0.45 removes overlapping detections.

YOLOv8n Model Specifications:

Parameter	Value
Input Resolution	640 x 640 px (letterboxed)
Parameters	3.2 million
FLOPs	8.7 billion
mAP@0.5 (COCO)	37.3%
Model File Size	6.3 MB (.pt)

Parameter	Value
ONNX Exported Size	~12 MB (.onnx, FP32)
INT8 Quantised Size	~3 MB (estimated)

Table 6.7 -- YOLOv8n model specifications.

6.2.3 HSV Colour-Based PPE Verification Algorithm

Since the generic COCO-trained YOLOv8n model does not contain 'helmet' or 'vest' as native classes, the system implements a supplementary colour-based PPE verification algorithm that operates on anatomically segmented worker regions.

Pseudocode:

```
FUNCTION verify_ppe(frame, worker_boxes):
    hsv_frame = BGR_to_HSV(frame)
    FOR EACH worker IN worker_boxes:
        box = worker.bounding_box
        head_region = box[y1 : y1 + 0.25*height, x1 : x2]
        torso_region = box[y1 + 0.25*height : y1 + 0.70*height, x1 : x2]

        // HELMET CHECK
        yellow_mask = inRange(head_region, [20,80,100], [35,255,255])
        orange_mask = inRange(head_region, [5,120,120], [18,255,255])
        white_mask  = inRange(head_region, [0,0,190],   [180,40,255])
        helmet_ratio = count_nonzero(OR(yellow, orange, white)) / area
        IF helmet_ratio > 0.14:
            EMIT detection("safety-helmet", confidence=0.65+ratio)

        // VEST CHECK
        vest_yellow = inRange(torso_region, [22,90,100], [40,255,255])
        vest_orange = inRange(torso_region, [6,120,110], [19,255,255])
        vest_ratio = count_nonzero(OR(vest_yellow, vest_orange)) / area
        IF vest_ratio > 0.18:
            EMIT detection("safety-vest", confidence=0.60+ratio)
    RETURN equipment_list
```

PPE Item	Region	HSV Lower Bound	HSV Upper Bound	Min Ratio
Yellow Helmet	Head (top 25%)	[20, 80, 100]	[35, 255, 255]	0.14
Orange Helmet	Head (top 25%)	[5, 120, 120]	[18, 255, 255]	0.14
White Helmet	Head (top 25%)	[0, 0, 190]	[180, 40, 255]	0.14
Yellow Vest	Torso (25-70%)	[22, 90, 100]	[40, 255, 255]	0.18
Orange Vest	Torso (25-70%)	[6, 120, 110]	[19, 255, 255]	0.18

Table 6.8 -- HSV colour thresholds for PPE verification.

6.2.4 Alert Debounce Algorithm

To prevent alarm fatigue from continuous violations, the system implements a per-worker, per-violation-type debounce mechanism. The algorithm maintains a hash map of (worker_id, violation_type) -> last_alert_timestamp. A new alert is triggered only when the elapsed time exceeds the configurable cooldown period (default: 6 seconds).

```
FUNCTION should_trigger_alert(worker_id, violation_type, current_time):
    key = (worker_id, violation_type)
    last_time = debounce_cache.get(key, 0)
    IF (current_time - last_time) >= COOLDOWN_SECONDS:
        debounce_cache[key] = current_time
        RETURN True
    RETURN False
```

6.2.5 Privacy-Preserving Facial Anonymisation Algorithm

The privacy module applies Gaussian blur to the estimated head region (top 28% of each worker bounding box) with a dynamically computed kernel size. This ensures anonymisation without requiring a dedicated face-detection model, reducing computational overhead on edge hardware.

```
FUNCTION apply_face_blur(frame, workers):
    FOR EACH worker IN workers:
        box = worker.bounding_box
        head_y2 = box.y1 + 0.28 * (box.y2 - box.y1)
        face_roi = frame[box.y1 : head_y2, box.x1 : box.x2]
        ksize = max(15, (face_roi.width / 3) | 1)   // ensure odd
        blurred = GaussianBlur(face_roi, (ksize, ksize), sigma=30)
        frame[box.y1 : head_y2, box.x1 : box.x2] = blurred
    RETURN frame
```

6.2.6 Synthetic Worker Detection (Demonstration Mode)

When no physical camera is connected, the system generates animated construction-site scenes and detects worker positions using skin-tone contour analysis. This enables full-pipeline demonstration without hardware dependencies.

```
FUNCTION detect_synthetic_workers(frame):
    hsv = BGR_to_HSV(frame)
    skin_mask = inRange(hsv, [8, 50, 130], [25, 180, 240])
    contours = findContours(skin_mask, RETR_EXTERNAL)
    workers = []
    FOR EACH contour IN contours:
        area = contourArea(contour)
        IF 150 < area < 1500:
            (hx, hy, hw, hh) = boundingRect(contour)
            cx, cy = hx + hw/2, hy + hh/2
            worker_box = extrapolate_body(cx, cy, width=56, height=175)
            workers.append(worker_box)
    equipment = verify_ppe(frame, workers)   // reuse HSV PPE algorithm
    RETURN workers, equipment
```

6.2.7 Computational Complexity Summary

Algorithm Stage	Time Complexity	Dominant Factor
CLAHE Enhancement	O(W x H)	Frame pixel count
YOLOv8n Inference	O(N)	Grid cells x anchors (fixed by architecture)
HSV PPE Verification	O(K x W_r x H_r)	K workers x region pixel area
PPE-Worker Association	O(K x E)	K workers x E equipment items
Violation Analysis	O(K x R)	K workers x R rules

Algorithm Stage	Time Complexity	Dominant Factor
Alert Debounce	O(1) amortised	Hash-map lookup
Facial Blur	O(K x W_f x H_f)	K workers x face region area
JPEG Encoding	O(W x H)	Output frame pixels

Table 6.9 -- Computational complexity of each pipeline stage.

6.3 About Flowchart

This section presents three flowcharts that together describe the complete data and control flow of the Edge-Enabled Privacy-Preserving Construction Safety System. Each flowchart corresponds to a different abstraction level: (1) the high-level system processing loop, (2) the internal edge pipeline module interconnections, and (3) the frontend-backend data flow architecture.

6.3.1 System Processing Flowchart

Figure 6.1 illustrates the main system processing loop from camera input through AI detection, safety analysis, voice alert generation, database storage, and privacy-guarded frame annotation. The loop continues until monitoring is explicitly stopped by the operator via the dashboard.

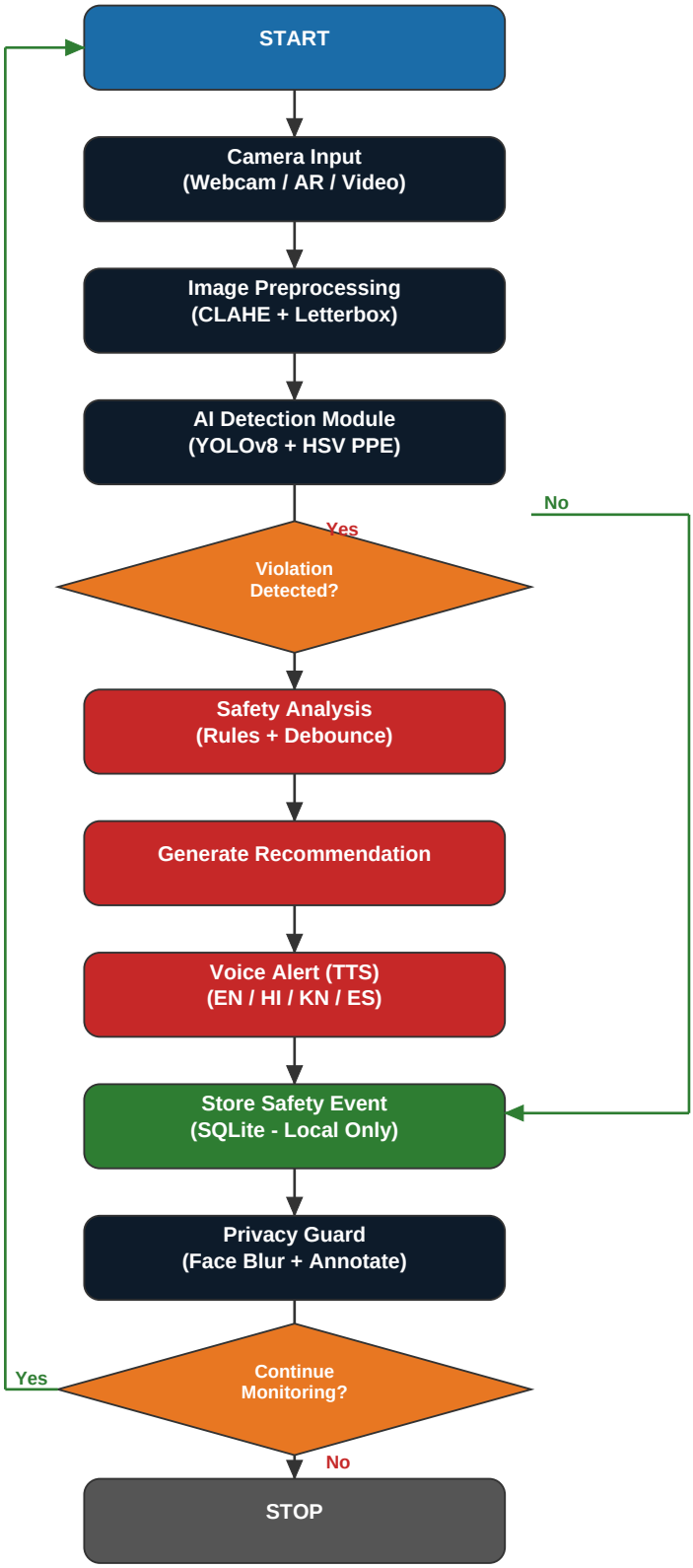

Figure 6.1 -- System processing flowchart: Camera input through detection, analysis, alerting, and privacy-guarded output. Green arrows indicate the 'No violation' and loop-back paths.

6.3.2 Edge Processing Pipeline Flowchart

Figure 6.2 shows the internal module-level pipeline within the backend. Frames flow left-to-right through capture and enhancement, then right-to-left through AI detection and alert generation, and finally left-to-right again through privacy guard and browser streaming. The SQLite database receives events as a side-channel from the Violation Analyser.

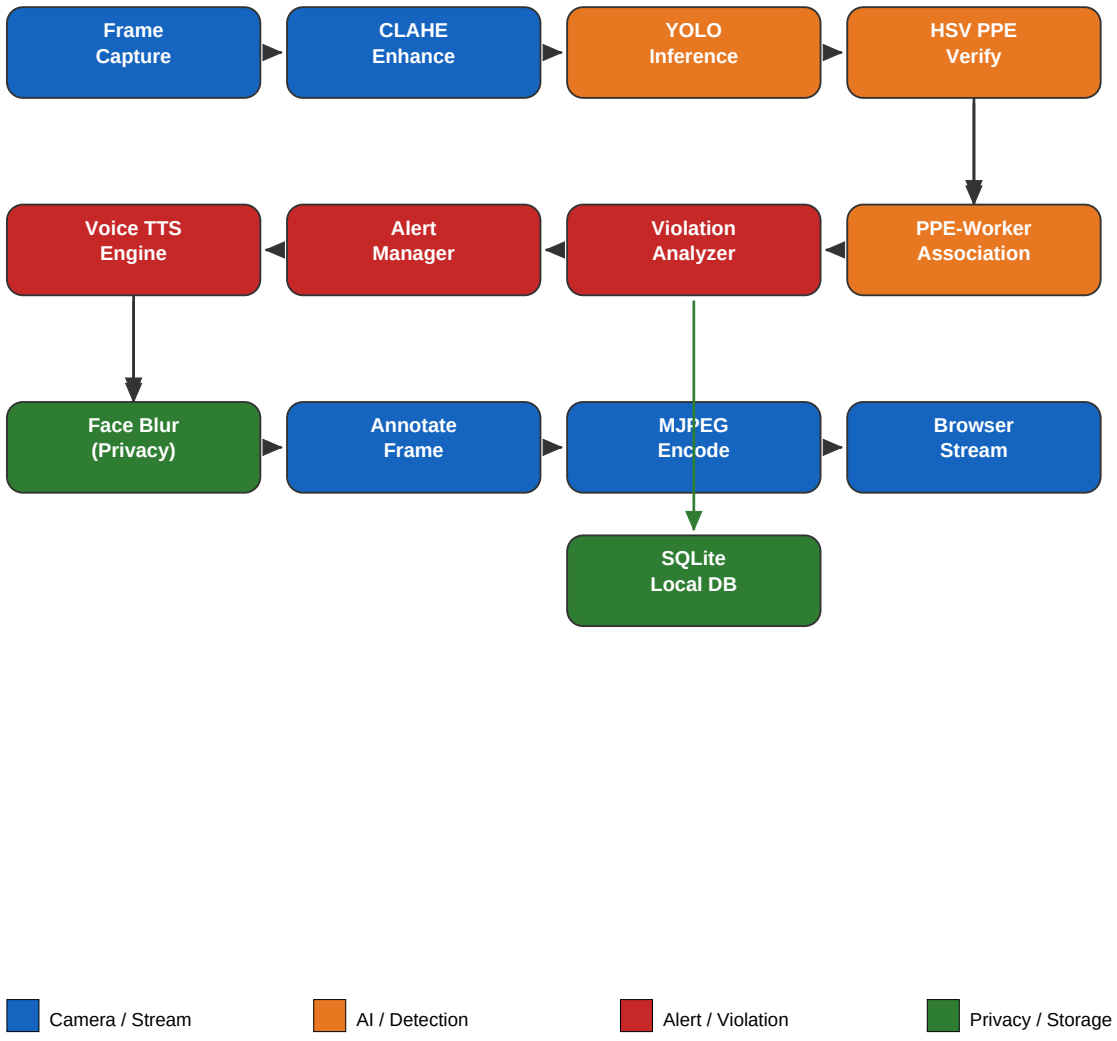

Figure 6.2 -- Edge processing pipeline: module-level data flow with colour-coded functional categories (blue = camera/stream, orange = AI, red = alerts, green = privacy/storage).

6.3.3 Frontend-Backend Data Flow Diagram

Figure 6.3 depicts the separation of concerns between the React frontend, the FastAPI REST API layer, and the backend processing modules. All data exchange occurs over HTTP on the local network. No direct database access or AI computation exists in the frontend. The bottom row shows the edge hardware peripherals accessed exclusively by the backend.

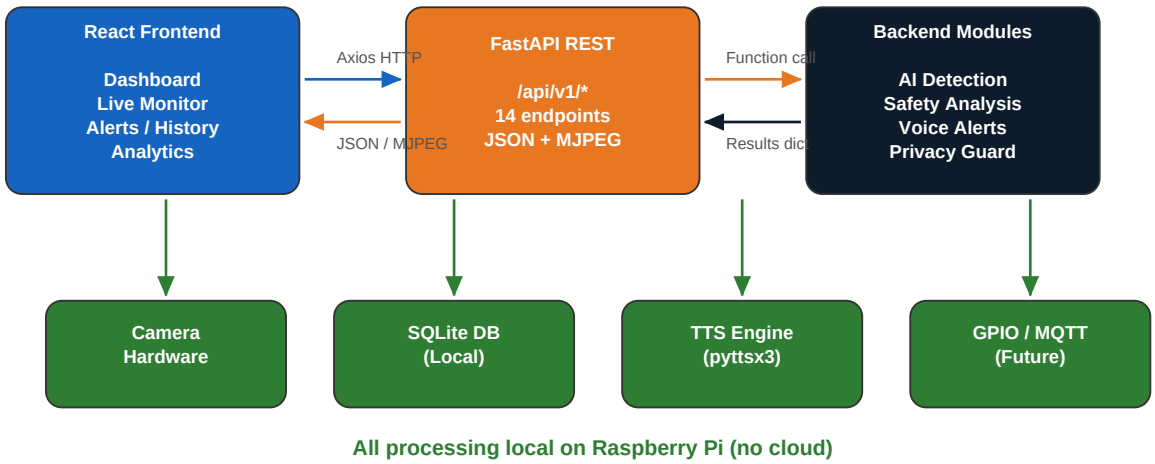

Figure 6.3 -- Frontend-backend data flow architecture. Bidirectional Axios/HTTP communication over local LAN. All processing remains on the edge device.

6.3.4 Flowchart Symbol Legend

Symbol	Shape	Meaning
Rounded Rectangle	Process	A processing step or module execution
Diamond	Decision	Conditional branch (Yes/No evaluation)
Arrow	Flow	Direction of data or control flow
Colour: Navy/Blue	---	Core processing or stream modules
Colour: Orange	---	AI detection and inference modules
Colour: Red	---	Violation detection and alert modules
Colour: Green	---	Privacy guard and local storage modules
Colour: Grey	---	Terminal states (Start / Stop)

Table 6.10 -- Flowchart symbol and colour legend.